https://github.com/TSika-1/CS352-Unit1-Assignment2

Step 4: How I tested one round

To test my round, I made a line of 5 contestants (Contestant0 through Contestant4) and printed the line after every move. I checked each printed line by hand against the table in the assignment. Contestant0 answered correctly and stayed at the front. Contestant1 and Contestant2 answered incorrectly and each moved to the end. Contestant3 and Contestant4 answered correctly and each moved to the front. The line ended as Contestant4, Contestant3, Contestant0, Contestant1, Contestant2, which is what I got when I worked it out by hand. The line always had 5 contestants, with none missing or repeated. Because every step matched my hand check, I concluded that one round works correctly.

Step 5: How I tested fairness

To check that the algorithm isn't biased, I ran 8 contestants for 12 rounds and recorded each contestant's position at the end of every round. With 8 contestants the positions go from 0 to 7, so a fair average is about 3.5. My averages were 2.67, 2.83, 3.08, 3.25, 3.58, 4.00, 4.25 and 4.33. They are all fairly close to the middle, and the lowest and highest are both about one position from 3.5. No contestant had an average near 0 or near 7, so nobody got stuck at the front or the back. The averages are not all exactly 3.5 because there were only 12 rounds, so some luck is normal. They should move closer to 3.5 with more rounds. This makes sense because every contestant has the same 50/50 chance each round.

Step 8: A different algorithm

The organizers don't like moving people all the way to the front or the end. A different algorithm could swap each contestant with another contestant at a random position in the list. To implement this, I would only need to change the playRound() method. The Contestant class, the coin flip, the position recording and the timing code could stay the same. Instead of remove() and add(), I would use get() and set() to swap two contestants. This should be easy to implement because it is a small change in one method.

I expect the new version to run faster. My results show how slow the current version gets. With 43 contestants, 10,000 rounds took only 28 ms. With 45,000 contestants, just 1,000 rounds took about 168 seconds. That is far slower than the larger list size alone would explain. This happens because remove() and add(0, ...) make the ArrayList shift many elements every time, and there are more elements to shift in a big list. Swapping with get() and set() doesn't shift anything, so

the time per round should grow with the number of contestants instead of much faster. The new version would help the most with 45,000 contestants and make little difference with 43. I would repeat the fairness test to make sure the new algorithm doesn't favor certain positions.

Timing table (Steps 6 and 7)

Contestants	Rounds	Time (ms)
43	10	0
43	100	1
43	1,000	9
43	10,000	28
45,000	10	1,793
45,000	100	17,165
45,000	1,000	168,493
45,000	10,000	*(still running)*